

FEILIAN: technical assessment of the submitted specification and implementations

Cryptographic review · Version 1.0.0

Mounir IDRASSI (mounir@amcrypto.jp)

24 September 2026

Abstract. This review examines the FEILIAN hash submission dated 30 June 2026, including its specification, six C implementations, four RTL architectures and validation material. Static analysis identifies a message-length binding defect in the 1SC, 4SC and 8SC hardware wrappers; the 2SC counter path differs. The specification and implementations disagree on security-relevant details. All six Appendix B examples are exactly reproduced: the 512/768 examples use cumulative padded-block lengths, while the 1024 examples use actual message lengths. Verification covers all 45 printed round states and nine initial states. Ordinary tests pass all 24,582 C-to-KAT comparisons. Exact arithmetic confirms the Sigma maps' invertibility and intended constants. Shared test-generation code also has portability and error-propagation defects, distinct from the hash primitive. The security discussion does not establish the claimed composition, margin or application guarantees. No full-round cryptanalytic break of the correctly encoded software hash is established. A corrected, unambiguous submission and renewed independent review are recommended.

Prior work and preparation. Markku-Juhani O. Saarinen's FEILIAN report of 23 September 2026, credited there with AI assistance, was consulted before this review. It reports silent allocation failure in the reference C implementations. R5 extends that source-level observation to the optimized implementations; it is not presented as an independently discovered original finding. R1-R4 and R6-R8 describe additional observations relative to that report as consulted on 24 September; no exhaustive novelty claim is made. A second researcher subsequently supplied a private review. That researcher is credited for the Appendix B counter explanation and for raising the additional hardware and shared-DRNG issues incorporated after critical checking. The second review is not endorsed as a whole; its broader experimental claims are not included in this report's verified coverage. The provenance record identifies the reviewed files and the limits of verification.

AI-use disclosure. OpenAI Codex assisted with source and specification analysis, mathematical checks, verification code and publication preparation. The executable coverage and the static conclusions are reported separately. This package is prepared for Mounir IDRASSI's review and possible release; preparation does not assert that he has independently verified every conclusion. See AI_DISCLOSURE.md.

Assessment and scope

Recommendation: withhold a positive security assessment and integration approval for this submission. There is a high-severity message-length encoding defect in three supplied hardware implementations, several mutually inconsistent definitions of the hash, and substantial gaps in the argument supporting its security claims. These are sufficient grounds to request a corrected submission and renewed review. This review does **not** establish a full-round cryptanalytic break of the hash defined by the written compression algorithm or of the normally operating C implementations.

The most consequential concrete finding beyond the existing allocation-failure report is at the boundary between zero padding, the bit counter, and the RTL compression interface. The 1SC, 4SC, and 8SC cores omit the current block's length from the compression input. The consequence is loss of injective message encoding, independently of how strong the ARX compression function is. The 2SC core contains a different, corrected counter path.

The principal source is Specification.pdf, dated 30 June 2026. Page references below are the **printed page numbers**; the PDF viewer's page number is one greater. Its SHA-256 is 8e3a9a109f3188ab57cbd0156c243f80a6453051cf98006dfef11d66bdeb5fa8. The input manifest fingerprints all 74 original files. The archive digest on the public report was not checked against an original ZIP, which was not present here.

ID	Finding	Assessment	Evidence
R1	Current-block length omitted by 1SC/4SC/8SC RTL	High severity for those hardware hash implementations	Static dataflow and state-machine analysis; no RTL simulation
R2	Chapter 2, C, and RTL do not define one consistent algorithm	Blocks conformance and transfer of security conclusions	Source comparison, PDF rendering, independent model, exact constant checks
R3	Appendix B uses padded counts for 512/768, true counts for 1024	Security-relevant encoding inconsistency; C/KATs use true counts	All six digests, 45 round snapshots, nine initial states and nine domains verified
R4	C bit-input and length-conversion contracts are incomplete	Correctness/security defect; platform-dependent severity	Static analysis of all six C implementations
R5	Allocation failure is silently accepted in optimized C too	Extends the scope of the existing implementation finding	Static analysis; allocation failure not induced
R6	Proof and security-margin claims are not established	Major assurance gap, not a demonstrated hash break	Mathematical and compositional review
R7	MAC/KDF/XOF recommendations exceed what was established	Application specifications need separate review	Domain, entropy, interface, and proof-scope analysis
R8	Shared DRNG and KAT driver have portability and error-handling defects	Test-generation portability and error handling	Static source review and identical-file hashes; no failure injection

R1. Hardware message-length binding

Section 2.3, pp. 12–13, updates t to include the current block **before** calling the compression function. This is a security requirement because the message is padded with zeros and has no in-block delimiter or encoded length.

In the 8SC core, `cf_domain` is made from `hashed_bits_reg`. That register represents only the completed previous blocks. Its increment occurs in the `WAIT_CF` completion branch at line 160, after the compression output has already been calculated. The last-block flag is provided correctly, but the current final-block length is absent from the domain used to obtain the final digest. The compression module latches the supplied domain on start, so a later counter update cannot repair the result; see `compression.sv`.

The same defect occurs in the 1SC core and 4SC core. Their domain assignments are at line 135 and completion-time counter updates at line 157. Neither interface supplies a separately authenticated final-block length to compression.

Zero padding is not injective by itself. A final flag distinguishes the role of a block, but does not distinguish all possible logical lengths within that role. Removing the actual final length therefore permits distinct logical inputs to have an identical encoded compression transcript. No differential property, round reduction, birthday search, or allocation failure is needed for this structural failure. It affects variable-length hashing in these wrappers during normal execution. It should be treated as a collision-resistance failure of their message encoding, not as evidence that the specified ARX primitive has been cryptanalytically broken. Protocols imposing an independently authenticated, fixed input length would need a separate impact assessment.

The 2SC core is an important exception: it computes `hashed_bits_updated` from the current input length and passes that value into `make_domain`. Do not report this particular defect as affecting all four architectures.

The corrective requirement is to form one accepted block transaction containing the block, final flag, and cumulative bit count **including that block**, and preserve that tuple for the complete compression operation. Counter overflow and invalid final byte counts also require explicit rejection. The acceptance invariant is $t_{\text{compression}} = t_{\text{previous}} + \ell_{\text{current}}$. Checking only compression-round arithmetic will not detect an error in the wrapper that supplies this count.

This finding follows from the source’s dataflow. No RTL simulation was performed in this review. The source-level conclusion is strong; a corrected implementation still needs end-to-end RTL validation before deployment.

R2. Inconsistent algorithm definitions

There are independent discrepancies, so correcting one typo is insufficient.

Detail	Chapter 2 / specification	Supplied implementation evidence
IV word 7 AddConstant placement	0x3D84D5B5B5470917, p. 3 Tweak in row index 1, constants in row index 3, p. 9	C and RTL use 0x3F84D5B5B5470917 All six C versions and all four RTL round implementations use constants in row index 1 and tweak in row index 3
Counter word order	$t = t_0 \parallel t_1$; significance should be made explicit alongside the little-endian rule	C and 2SC use high word, then low word; 1SC/4SC/8SC use low word, then high word

Detail	Chapter 2 / specification	Supplied implementation evidence
Count supplied to compression	Includes the current block, p. 13	C and 2SC do; 1SC/4SC/8SC do not
Hardware variant labels	README labels 1SC and 4SC as 512-bit	Every supplied feilian_pkg.sv sets DIGEST_BITS = 1024 and VERSION = 0x400

The AddConstant discrepancy was checked against a rendered PDF page, not inferred solely from text extraction. The scalar implementation's round functions are explicit. Appendix A's C listing also follows the implementation's placement, so the disagreement exists within the PDF itself.

The C counter helper stores the high word in `hash_bits[0]` despite later comments identifying that slot as the low word. The SIMD `_mm256_set_epi64x` call agrees with the executed C ordering; some accompanying comments do not. Correcting the RTL count timing without also settling word order will leave a conformance failure.

Sage interval arithmetic confirmed that the C IV matches the first 256 hexadecimal fractional digits of π , including the 3F word. Exact integer square-root checks reproduced all four round constants. This makes the printed 3D value look like an erratum, but the submitters must designate the canonical algorithm. The text's description of these IV values as decimal digits is also inaccurate.

An independent Python implementation, using Chapter 2's IV and AddConstant placement, little-endian message words from §3.2, and the C high-word-first counter convention, produced different hashes from the C code for the empty message, `abc`, and 129 repetitions of `a`, for every digest size. This is a conformance comparison on ordinary messages. For example, the first 16 digest bytes for FEILIAN-1024(`abc`) are:

Interpretation	First 16 digest bytes, hexadecimal
Supplied scalar/SIMD C	0507e4b1ef75d025dfa303ea0a8e2c47
Chapter 2 under the stated decoding conventions	8b27103d98eabd06b36a8a1d2148d8e6

Security analysis must identify which version it covers. Tweak placement and counter ordering affect the actual permutation family, including which symmetry conditions correspond to valid message lengths. They cannot be dismissed as output formatting differences.

The hardware digest formatting and MMIO word order need an explicit byte-level contract as well. The cores reverse bytes within words and reverse word positions when forming the packed digest, while the MMIO port reads increasing packed slices. A driver may compensate, but no such driver contract or end-to-end testbench is supplied.

The top-level modules declare `DIGEST_BITS` but instantiate core `u_core` without forwarding that parameter. The package's `VERSION` also remains fixed, and the digest formatting and MMIO loops use the fixed sixteen-word state width. Changing only the top-level digest width therefore does not select FEILIAN-512 or FEILIAN-768 correctly. Width propagation, version selection, digest selection and slice bounds must be specified and reviewed together.

The hardware input interface supplies a byte count, with no partial-bit count. Its supported domain is therefore byte-aligned messages; this is an interface limitation, not evidence of mishandling a bit-length argument that the hardware never accepts. The eight-bit `valid_bytes` field also lacks rejection of final-block values greater than the 128-byte block capacity. For non-final blocks the helper always adds 1,024 bits. The 2SC path uses the final count before compression, whereas the other three defer it as described in R1. A legal range, empty-message convention and rejection behavior must be defined at the accepted-transaction boundary. These are static observations; no out-of-range request was executed.

Reset and bus-handshake behavior also need an explicit integration contract. A datapath register initialized on an accepted start before producing valid output is not defective merely because it lacks reset. A reset-required completion state or level-sensitive start is likewise not, by itself, a security vulnerability. These observations do not add separate reset or handshake findings.

R3. Appendix B uses two different counter conventions

All six supplied C implementations passed every entry in their corresponding KAT_2_12 file: 4,097 lengths, from 0 through 4,096 bits. That is 24,582 successful C-to-KAT comparisons. The independent Python model matched 20 selected KAT cases per version, covering partial bytes and block boundaries, and all nine additional ordinary example computations. Those C implementations use the cumulative true message length.

The second reviewer identified an exact explanation for the four previously unexplained Appendix B mismatches. Keeping the C IV, round operations, version selection, high-word-first counter and digest serialization, the following counts reproduce every printed example:

Variant	abc: final count (bits)	129 bytes of a: first / final count (bits)	Matching convention
FEILIAN-1024	24	1,024 / 1,032	Cumulative true message length
FEILIAN-768	1,024	1,024 / 2,048	Cumulative padded-block length
FEILIAN-512	1,024	1,024 / 2,048	Cumulative padded-block length

This was independently checked with the earlier review's model, against a fresh extraction of the hash-identified PDF. All **six digests, 45 printed round snapshots and nine initial states** agree under the assignments above. The check also recovers all nine domain values from the first-round states and confirms their high-word-first counter layout. It enforces the stated assignment for each variant, rather than merely accepting whichever convention happens to match. The complete evidence, numeric transcriptions and bounded ordinary-example checker are included.

The four 512/768 digests still disagree with C. For example, FEILIAN-512(abc) begins b69fc86f312df402fd5d243c61ab3768 in Appendix B and fc7320c565f60ec6a6ea26585f7c7d1f in C. What is now established is the precise numerical explanation. The historical generator version and reason for this discrepancy remain unconfirmed until the submitters provide provenance.

This is security-relevant encoding, not simply a different display convention. Padded-block counts do not bind the actual final logical length under zero-only padding. The

final flag does not restore that missing information. Any argument requiring injective message encoding cannot be transferred to the convention illustrated by the 512/768 appendix examples. This observation concerns the appendix convention; it does not show that the normally operating supplied C/KAT implementation uses that convention, and it does not establish a full-round cryptanalytic break of correctly encoded FEILIAN.

The initial IV matrices are displayed transposed relative to the numeric row-major convention. The computed round states and the second-block initial state are row-major. A uniform transpose of every displayed matrix would be incorrect. The checker normalizes only the initial IV display.

The correction must retain actual message-length binding and regenerate all examples from the designated algorithm. Treating padded and true counts as equally acceptable options would leave the security problem unresolved. Agreement with KATs generated from the same implementation demonstrates reproducibility, not independent agreement with the specification or a valid security proof.

R4. Bit-input and length contracts in C

The same padding logic appears in all six C implementations. In the scalar padding function, `memcpy` copies the complete final byte, but the unused bits of a partial final byte are not masked. Those bits subsequently enter the compression function. Either the API must expressly require a canonical final byte and reject violations, or the implementation must mask the bits outside `msg_len_bits`. The current header states only that the argument is the number of message bits.

This is a failure to make the result depend only on the declared bitstring. It is not evidence of a compression-function collision. The KAT generator's DRNG masks unused bits itself, which explains why the supplied partial-bit KATs do not test this API boundary; see `drng.c`.

There is a separate allocation-size arithmetic issue. The code adds the byte-rounded message length to the independently byte-rounded padding length. For a non-byte-aligned message, that allocates one byte more than the necessary whole-block buffer. This is harmless over-allocation by itself; it should not be misreported as a proven out-of-bounds access. The missing final-byte mask remains significant.

The public API accepts unsigned long long bit lengths, then casts them to `size_t` without checking representability. On a platform with narrower `size_t`, the declared message length can be silently truncated. This is a potentially serious correctness defect for a supposedly portable implementation. On platforms where the widths match, the subsequent `msg_bits + 7` still needs an overflow check. These conclusions are static; no 32-bit binary or extreme-length request was run.

The presence of a two-word counter does not make this one-shot API support the specification's entire domain of messages shorter than 2^{128} bits. It needs a streaming interface with validated counter updates, or an expressly smaller supported-input limit. A streaming implementation also avoids allocating a second copy of the complete message.

Output-length semantics need clarification too. The instance fixes `FEILIAN_VERSION` at compile time but accepts any positive `digest_len_bits` up to its maximum and truncates the digest. A shortened result from the FEILIAN1024 implementation is therefore a truncation of the 1024-version hash, not the independently versioned FEILIAN512 hash. That can be a legitimate API choice, but it must not be confused with

selecting another family member. Reject unsupported named lengths or expose raw truncation and family selection distinctly. For a permitted non-byte-aligned output length, a buffer of $\lceil t/8 \rceil$ bytes with unused bits cleared is a normal representation of a t -bit digest; storage rounding alone is not a vulnerability.

R5. Allocation-failure handling in optimized C

Saarinen’s 23 September report identifies reference implementations returning successful all-zero digests after padding allocation fails. Static inspection shows the same path in the SIMD implementation: the internal routine clears the digest and returns without an error status, and the public wrapper returns success. The 512- and 768-bit optimized sources use the same logic.

The fix must propagate failure through the public return code and leave the output unusable as a successful digest. Streaming reduces the memory-pressure exposure but does not replace correct error propagation. No allocation failure was induced during this review, and the public report’s witness was not rerun.

R6. Mathematical and security arguments

FEILIAN uses a 1,024-bit chaining state, a 1,024-bit message block, and a public tweak containing version, final flag, and a 128-bit counter. Its five phases each comprise one message-bearing round and three message-free rounds. Each full round has two SubColumn/ShiftRow layers. Compression applies a Davies–Meyer feedforward, $h' = E_{m,tw}(h) \oplus h$.

This architecture is intelligible and uses established ingredients. Its security still depends on the concrete message- and tweak-dependent permutation family. Naming HAIFA, Davies–Meyer, or Even–Mansour does not establish that family behaves like the ideal object used in a theorem.

First, §4.3.1, pp. 19–20, proves a statement about a compression interface with a **salt** and final padding that encodes message length. The preceding FEILIAN definition instead has a **final flag**, no salt, and zero padding without an in-block length field. The simulator validates the former padding and does not model the actual flag. The proof therefore does not establish the stated result for the specified construction. The original HAIFA paper describes explicit padding and salt conventions; adopting its name does not transfer every result to a changed encoding.

There is a plausible repair direction, which should be distinguished from a demonstrated mode failure: with an exact, non-wrapping bit count, a correctly authenticated final flag, canonical partial bytes, and a defined empty-message case, the written message-to-transcript encoding is injective. Unequal total lengths differ in the final count; equal-length distinct messages differ in their data. Zero padding is therefore not intrinsically fatal to the written design. R1 is the concrete case where an implementation removes an essential part of that argument.

Second, even within its stated ideal model, the simulator description is incomplete. Its accounting for unreachable queries, later connections into the reachable graph, and out-of-order queries is not sufficient to justify the claimed perfect consistency. The stated B2 event also needs to distinguish the public IV from guessed unknown states. The query measure must account for the total number of compression evaluations or processed blocks: one variable-length hash query can process many blocks. A complete theorem needs an exact interface, game, simulator, resource measure, and bad-event proof.

Third, the displayed advantage bound $O(q^2/2^{1024})$ becomes non-informative around the 512-bit birthday scale. Even a correct version of that bound would not, by itself, establish a 1,024-bit preimage or second-preimage claim. Those properties need their own bounds and input-length/resource assumptions. Conversely, the birthday-scale limit of this proof is not itself a preimage attack; these are different statements.

Fourth, §4.4 changes idealizations. The Black-Rogaway-Shrimpton analysis gives black-box results for block-cipher-based hashing under its stated model. It does not prove that FEILIAN's repeated-message ARX structure is an ideal cipher, or automatically identify Davies-Meyer compression with the independent random compression oracle assumed by §4.3.1. The acknowledged Davies-Meyer fixed-point property reinforces the need to keep those interfaces and assumptions distinct. For a freely chosen compression state, its cost is comparable to one inverse of the pre-feedforward permutation, not a 2^{512} birthday search. Reaching such a state from the fixed hash IV under valid counters, or establishing a fixed point of the separate XOF mode, are different questions. No such reachability result or witness is established here. No complete composition argument is provided.

Fifth, the component evidence does not quantify a full-compression security margin. The statistical tests in §4.2 and average single-bit avalanche table in §4.5.3 measure useful implementation properties, but cannot certify collision or preimage complexity. They do not establish bounds on worst-case structured differences, differential hulls, related message/tweak behavior, or full-round attacks. NIST itself cautions against using its SP 800-22 statistical suite as a cryptographic security assessment.

The critical composition questions are:

- The same message is injected five times, with no expansion or word reordering. Security analysis must include those correlated inputs; analyzing message-free permutations expressly leaves out that freedom.
- Constants repeat within each phase and the first and fifth phase have the same constant pattern. The phases are not independent random permutations.
- Column-wise SubColumn and row shifts retain a structural relationship with cyclic column permutations. Rotating the constants between phases makes joint analysis of the message and tweak essential. Whether a related parameter setting is a valid hash invocation depends on version, flag, length, and counter-word conventions.
- Reaching an attacker-selected internal state from the fixed IV is a real constraint, but the assertion that this necessarily costs at least a birthday search is not a general theorem. Likewise, secrecy of a chaining state alone does not prove every keyed application secure.
- The final message injection occurs in round 17, leaving seven SubColumn/ShiftRow layers after that injection. That is not evidence of an attack, but it explains why counting all twenty rounds is an incomplete way to state the margin against message-dependent methods.

The discussion of slide attacks in §4.6.2 correctly recognizes repeated structure but then appeals to diffusion and nonlinear operations making a slid relation improbable. Those properties alone do not exclude structural slide relations. The meet-in-the-middle and boomerang discussions similarly assert adequate phase counts without supplying quantitative bounds or reproducible supporting searches. These are hypotheses to investigate, not established exclusions.

There are also checkable errors in the security text. On p. 25 the displayed boomerang criterion is $p^2 q^2 > 2^{2n}$, which is impossible for probabilities. This was

visually verified in the PDF. Correcting the missing minus sign would not resolve the more general issue: the appropriate random baseline and the use of p^2 q^2 require a precisely defined experiment and justification of dependencies. Reference [9], cited for a hash second-preimage argument on p. 20, is a paper about KeeLoq rather than the claimed hash result. The unsupported numerical claims and mis-citation should be corrected, not used as security evidence.

There are also positive mathematical checks. Both Sigma maps have rank 64 over $\text{GF}(2)$. Representing rotation by x , the word modulus is $x^{64} + 1 = (x + 1)^{64}$; each three-term Sigma polynomial takes value 1 at $x = 1$, so it is coprime to that modulus and invertible. Sage independently confirmed the gcds and one-dimensional fixed spaces. These facts do not rely on one rotation amount being prime. They establish linear-map properties, not differential probabilities through modular additions.

The SubColumn inverse in §4.5.2 is algebraically consistent with the forward equations, and 10,000 deterministic random 256-bit inputs passed a forward/inverse check. The algebraic inverse, together with invertible row shifts and XOR injections, establishes that the round composition before feedforward is a permutation for fixed message and tweak. There is no rank defect in either Sigma map. The probability-one component differentials acknowledged by the specification remain relevant to security analysis, but their existence alone does not establish a full-hash weakness. No independent exhaustive differential search or full-round attack construction was performed here.

The generic-security discussion should distinguish the variants. Only the 1,024-bit output version has a chaining state equal in width to the digest. The 512- and 768-bit versions retain a 1,024-bit state internally. A collision in a truncated output is not necessarily an internal-state collision, so generic multicollision and herding costs cannot be transferred between these cases merely by substituting the digest length. The fixed-salt HAIFA analysis also retains qualifications concerning multicollisions and herding; an unqualified claim of inheriting all HAIFA protections is too broad.

Classical and quantum claims

Table 4.1 explicitly labels its claims as classical. Under ideal behavior, its collision and preimage exponents are the usual ones for the three output lengths; their achievement by FEILIAN is what remains unestablished. The generic quantum query scales are different:

Digest bits	Classical collision exponent	Classical preimage exponent	Generic quantum collision exponent	Generic quantum preimage exponent
512	256	512	about 170.7	256
768	384	768	256	384
1,024	512	1,024	about 341.3	512

The quantum columns use the usual idealized collision-search and preimage-search query models, from Brassard-Høyer-Tapp and Grover. They are not practical gate-count or memory estimates and are not newly discovered attacks on FEILIAN. The specification's claim that there is no better dedicated quantum method needs analysis of the actual construction; a classical simulator proof does not establish it. In particular, the 512/1,024-bit classical claim should not be restated as that many bits of quantum security.

R7. MAC, KDF, XOF and interface composition

The claim that the preceding indistinguishability discussion licenses general replacement of a random oracle is too broad even before the proof defects are considered. Ristenpart, Shacham, and Shrimpton show why ordinary indistinguishability does not automatically cover all security notions with multiple adversarial stages.

For the proposed prefix MAC, finalization is a sensible defense against ordinary length extension, but collision resistance alone does not prove MAC security. A useful bound must include key entropy, tag length, query counts, total processed data, and the allowed interfaces. A claim of 512-bit MAC security cannot hold independently of those parameters. A vetted MAC construction would still require a sound assumption about FEILIAN and a correct implementation.

Tag collisions among already queried messages are not fresh-message MAC forgeries. For an ideal random-function MAC with t -bit tags, the generic verification-guessing contribution is bounded by $q_v/2^t$, where q_v counts verification attempts. A PRF-based argument adds the relevant distinguishing advantage; see Bellare–Rogaway, Proposition 7.3. The $2^{t/2}$ birthday scale for tag collisions does not, by itself, reduce the generic forgery exponent to $t/2$. This distinction does not supply the missing proof for FEILIAN’s actual prefix MAC.

For the KDF, $\text{PRK} = \text{FEILIAN}(\text{IKM})$ does not establish a general extractor for arbitrary non-uniform sources. A deterministic function cannot guarantee uniformly distributed output for every high-entropy source; a random-oracle argument requires suitable independence, secrecy, and entropy conditions. Hashing does not create entropy. The proposed expansion also needs an unambiguous encoding of context and counter, an output limit, and explicit separation from other uses of the same secret. A fixed-length PRK and a fixed, known-width trailing counter give unambiguous parsing even when info is variable-length: the counter is the final fixed-width field. Specify its injective encoding, byte order, allowed iteration range starting at one and overflow rejection. A variable-width alternative needs its own unambiguous framing. The specification gap does not imply that every possible encoding is ambiguous. Calling PRK a salt is misleading: in that expression it serves as secret key material. This is not HKDF, whose extract step is HMAC-based and whose salt and context handling are specified in RFC 5869.

For the XOF, the distinct squeezing version values are a useful design choice. However, the mode changes the finalization rule during absorption and then applies compression to an internal root; it is not the hash mode analyzed in §4.3.1. It needs its own definition and proof. Specify the empty-block encoding, counter word order and bound, the output truncation for each variant, the initial root for an empty input, and behavior on counter exhaustion. A finite 128-bit counter cannot silently be allowed to wrap while retaining the claim that every squeeze call is distinct. No MAC/KDF/XOF implementation or corresponding KATs were present.

Keyed applications create additional implementation requirements. In 1SC, 2SC, and 4SC, the digest signal is continuously derived from the current chaining register, and the top-level MMIO read path is not restricted to completed hashes. Message registers are readable too. Whether this crosses a security boundary depends on who can access the peripheral; it is not an additional public-hash vulnerability by itself. It does mean the supplied public-hash interface must not be assumed to keep a secret prefix or intermediate keyed state isolated. Similarly, C heap copies of keyed inputs are freed without explicit erasure. A keyed API should define state exposure and clearing rather than inherit them accidentally.

The scalar and SIMD ARX arithmetic has fixed operation schedules and no obvious message-dependent branches or memory indices. That is favorable source-level evidence for timing behavior on the intended platforms. No machine-code timing certification, leakage measurement, fault analysis, or secret-state isolation validation was performed, so these are not established properties of a deployable MAC or KDF.

R8. Shared DRNG and test-driver assurance

The six `drng.c` files supplied with FEILIAN are byte-identical, as recorded in `shared_drng.json`. Its header attributes the software to ICCS, and the submission README marks the KAT generator and DRNG files as unmodifiable submission infrastructure. These observations support treating the defects below as a shared test-infrastructure issue and coordinating an upstream correction with the organizers. They are not defects in FEILIAN's compression mathematics.

The SM3 rotation macro, at line 36, allows a shift by the word width when the rotation count is zero. The SM3 round calls at lines 90 and 94 reach that case. With 32-bit unsigned `int`, this is undefined behavior under C's shift rules, regardless of whether a particular processor masks its machine-instruction shift count. Use defined 32-bit rotation semantics for every permitted count; see C11 draft N1570, §6.5.7 paragraph 3.

Error propagation is also incomplete. `SM3_DRNG_Instantiate` ignores both `SM3_df` return values at lines 252 and 257, and the KAT driver ignores the initialization status at lines 296, 379, 487 and 601. The failed state is **not necessarily all zero**. Failure of the outer allocation follows a context clear; an inner derivation failure can leave seed material or a padded state copied into the context before success is returned. The result depends on the failing stage and input. Propagate every initialization/derivation failure and stop generation before recording purported test vectors.

The partial-byte mask used by the generator is consistent with its retained-bit count; it is not an additional mask defect. The accepted seed-pointer contract should also be explicit: if a null pointer with zero length is supported, avoid passing it to `memcpy`; C's library pointer requirements still apply at zero length (§7.24.1 paragraph 2). The supplied normal KAT initialization passes a real seed array, so this is conditional API hardening, not a demonstrated normal-input failure.

These conclusions are static. No sanitizer run or allocation-failure experiment from the second review is claimed as independently reproduced. They affect the portability and reliability of regenerating vectors. They do **not** invalidate the already supplied message/digest pairs or the 24,582 comparisons performed here, which hash fixed KAT message bytes directly and do not execute the DRNG. Preserve those records while correcting and independently checking the generator.

Validation and limits

The verification driver can build the six supplied C sources from a separately obtained, hash-verified submission and checks canonical inputs only. The C conformance record and model record contain the results and complete ordinary-example digests. The appendix check verifies the six printed examples under their identified count conventions, all 45 round snapshots, nine initial states and nine recovered domains. The Sage checks perform exact polynomial and constant calculations, with a separate arithmetic record. The RTL source index identifies the relevant counter and variant locations; source locators also cover the hardware interface and shared test generator. Their source observations are not executable hardware or failure-path tests.

The package provides a self-contained Python check of ordinary examples and component arithmetic, plus an optional check against the supplied C sources and published short-message KATs. See REPRODUCING.md for commands, dependencies and coverage. The latter path verifies all 74 original file hashes before compilation. Original submission files are not redistributed.

The ordinary C tests passed. Compiler warnings concerned a validated positive output-length conversion and unused SIMD masks; those warnings were not treated as security findings. The long-message KAT sets (2^{23} , 2^{33}) and million-iteration loop were not run. No software source was patched, no noncanonical bit-input or failure-path witness was exercised, and no HDL execution was performed. Passing these checks is evidence about specified computations and internal consistency, not a proof of security.

Requested corrections

Request a corrected submission before spending substantial effort on full-round security claims. The immediate requests should be:

1. Designate one authoritative algorithm, settling the IV, AddConstant rows, state/byte order, counter word order, empty input, partial-bit convention, and all named output variants. Identify which exact definition produced each security experiment.
2. Correct the RTL length binding and variant definitions, and provide end-to-end tests across all four architectures against an independently implemented model. Compression-only synthesis results do not validate message framing or digest presentation.
3. Correct the C input-length checks, partial-byte handling, allocation error propagation, and output-length contract. Prefer a bounded-memory streaming interface with explicit counter-overflow behavior.
4. Regenerate all appendix and KAT material from the designated version using true message-length binding, with independently checked intermediate states and a reproducible provenance record. Explain the mixed appendix counter conventions.
5. Coordinate correction of the shared DRNG and KAT driver with the organizers: use defined rotations, propagate failures and independently check deterministic output across supported toolchains. Preserve existing vectors for comparison.
6. Replace the mode proof with one for the actual interface and obtain independent review of the complete message/tweak/state composition. State separately what is proved in an ideal model, what is experimentally supported, and what remains conjectural. MAC, KDF, and XOF recommendations need their own specifications and bounds.

The hardware encoding defect is the strongest concrete reason to stop use of the affected implementations now. The inconsistent algorithm definitions and unsupported security margins are the strongest reasons to withhold confidence in the submission as a whole. They justify a hold and a corrected review target; they do not justify claiming that this review has broken the full-round, correctly encoded software hash.

References

1. Lei Wang, Ling Song, Yaobin Shen, Kaixuan Wang, Zicheng Shi and Xin Yi. *FEILIAN algorithm specification*, 30 June 2026. The supplied 68-page PDF is identified by its digest above and in the input manifest. Printed page numbering is used throughout.
2. Markku-Juhani O. Saarinen. *FEILIAN (hash-10)*, 23 September 2026. Public report, consulted 24 September 2026.
3. Eli Biham and Orr Dunkelman. *A Framework for Iterative Hash Functions — HAIFA*. IACR ePrint 2007/278. Paper.
4. John Black, Phillip Rogaway and Thomas Shrimpton. *Black-Box Analysis of the Block-Cipher-Based Hash-Function Constructions from PGV*. CRYPTO 2002. Author-hosted paper.
5. NIST. *Decision to Revise NIST SP 800-22 Rev. 1a*, 19 April 2022. Notice.
6. Lov K. Grover. *A Fast Quantum Mechanical Algorithm for Database Search*. 1996. Paper.
7. Gilles Brassard, Peter Høyer and Alain Tapp. *Quantum Cryptanalysis of Hash and Claw-Free Functions*. 1997. Paper.
8. Thomas Ristenpart, Hovav Shacham and Thomas Shrimpton. *Careful with Composition: Limitations of the Indifferentiability Framework*. EUROCRYPT 2011. Paper.
9. Hugo Krawczyk and Pasi Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, May 2010. RFC.
10. ISO/IEC JTC1/SC22/WG14. *N1570: Committee Draft, Programming Languages — C*, 12 April 2011. §§6.5.7 and 7.24.1. Draft.
11. Mihir Bellare and Phillip Rogaway. *Introduction to Modern Cryptography*, course notes, 2005. Proposition 7.3. Author-hosted notes.

The original text of this report is licensed under CC BY 4.0. See LICENSING.md for software terms and third-party exceptions.